

PdfPecker manual

Fast HTML and Markdown to PDF for .NET, with no dependencies.

For PdfPecker 1.3.0 · pdfpecker.net/docs

Contents

PdfPecker 1.3.0 . . . . . **3**
- Install 3
- How it compares 3
- System requirements 5
- Usage 5
- Merging PDFs 10
- PDF to image 12
- What is supported 13
- Limitations 0
- Performance 0
- Building 0
- How it works 0

PdfPecker

Fast HTML and Markdown to PDF conversion, and PDF to PNG rendering, for .NET, with no dependencies.

PdfPecker has its own HTML parser, Markdown parser, CSS engine, layout engine and PDF writer, and its own PDF renderer. It does not start a browser, ship native binaries or pull in third-party packages; the NuGet package depends only on the .NET base class library.

```
using PdfPecker;

byte[] pdf = HtmlToPdf.Convert("<h1>Hello</h1><p>World</p>");
File.WriteAllBytes("hello.pdf", pdf);
```

Install

```
dotnet add package PdfPecker
```

Until the package is published to nuget.org, build it locally and add the folder as a package source:

```
dotnet pack src/PdfPecker -c Release -o ./nupkg
dotnet nuget add source "$(pwd)/nupkg" --name pdfpecker-local
dotnet add <your project> package PdfPecker
```

The package targets `net10.0` and works on .NET 10 and later.

How it compares

The same three documents converted to A4 PDF by PdfPecker and four free .NET alternatives, on one 4-core cloud VM (Intel Xeon @ 2.8 GHz, 16 GB, Linux, .NET 10.0.12). Times are medians after a 3-second warm-up; "first PDF" includes starting the browser where there is one. Parallel runs one conversion per core at once on the 5-page document. Peak memory is the proportional set size (PSS) of the whole process tree, so a browser's helper processes are included once and shared pages are not double-counted.

Converter	Engine	To deploy	First PDF	Invoice, 1 page	Feature tour, 5 pages	Report, 30 pages	Parallel	Peak memory
PdfPecker	own, managed	3.1 MB DLL	0.27 s	**0.86 ms**	**2.7 ms**	**39 ms**	**962 docs/s**	**83 MB**
HtmlRenderer.PdfSharp 1.6.1	own, managed (CSS 2)	2.4 MB of DLLs	0.47 s	5.0 ms	16 ms	473 ms	88 docs/s	255 MB

Converter	Engine	To deploy	First PDF	Invoice, 1 page	Feature tour, 5 pages	Report, 30 pages	Parallel	Peak memory
wkhtmltopdf 0.12.6	Qt WebKit, native	about 90 MB with Qt	0.31 s	289 ms	333 ms	693 ms	12 docs/s	164 MB
PuppeteerSharp 25.13	Chromium 141	about 600 MB browser	0.91 s	128 ms	166 ms	539 ms	12 docs/s	849 MB
Microsoft.Playwright 1.63	Chromium 141	browser + 135 MB driver	1.08 s	142 ms	191 ms	522 ms	11 docs/s	986 MB

How to read it:

- PdfPecker is 60 to 150 times faster than the Chromium-based converters on the 1 and 5-page documents and about 14 times faster on the 30-page report, and uses a tenth of their memory. Per-document overhead (a browser tab, a process start) dominates for the others on small documents.
- Output is not identical. Chromium is the reference for modern CSS and runs JavaScript, which PdfPecker does not. HtmlRenderer supports only HTML 4 and CSS 2 (it drops the invoice's table header colours), and wkhtmltopdf uses a 2016 WebKit, shrinks content to fit by default (24 pages for the report instead of 29) and was archived in 2023.
- PDF sizes for the 1 / 5 / 30-page documents: PdfPecker 12 / 26 / 126 KB, Chromium 72 / 139 / 171 KB, wkhtmltopdf 30 / 59 / 191 KB, HtmlRenderer 35 / 65 / 273 KB.
- Each converter was used the way its documentation suggests for many documents: PdfPecker and HtmlRenderer in process, wkhtmltopdf as one process per document, and PuppeteerSharp and Playwright with one shared browser and a new tab per document (Playwright in one shared browser context, tagged PDF output off for both).

Commercial converters were not measured here. One user compared PdfPecker with **IronPDF 2026.7.2** (Chrome-based) on the 26 templates of their production app, on a 22-core development machine after warm-up: IronPDF took 13.6 s for all 26 documents one at a time and PdfPecker 17 ms (about 800 times faster), and with 22 in parallel IronPDF made 9.2 documents/s and PdfPecker 1,143 (about 125 times). 24 of the 26 documents came out identical; the other two differ by less than 0.25 mm.

To reproduce the table (needs wkhtmltopdf on `PATH` and a Chromium at `CHROMIUM_PATH`):

```
dotnet run --project bench/PdfPecker.Comparison -c Release
```

The comparison project is not part of `PdfPecker.sln`, so its third-party packages never reach the library or its tests. Your numbers will differ with hardware and documents.

System requirements

Required: the .NET 10 runtime (or later). Nothing else: the library is pure managed code with no native binaries, browser or third-party packages, so it runs wherever .NET 10 runs (Windows, Linux, macOS; x64 and ARM64).

Recommended:

- Register the fonts your templates use with `options.Fonts` (see Fonts); other fonts fall back to Helvetica, Times or Courier metrics.
- Memory and CPU are small. Measured on a 4-core 2.1 GHz Xeon cloud VM (Linux, .NET 10, Release), each document in a fresh process:

Document	Pages	Time, warm	Time, first in process	Peak process memory	Smallest heap that works
Payslip, invoice, voucher	1 to 3	1 to 3 ms	0.2 s	50 MB	under 16 MB
Same with PDF/A or embedded fonts	1 to 3	3 to 5 ms	0.2 s	56 MB	under 16 MB
Report with tables	30	30 to 40 ms	0.3 s	75 MB	under 16 MB
Report with tables	1,000	1.7 s	3 s	480 MB	245 MB

  The process itself starts at about 32 MB; a conversion uses about one core, so throughput grows with cores. PDF/A and font embedding add a few milliseconds and about 5 MB for the embedded font data. Memory grows roughly with page count, so plan about 0.5 MB per page for very long documents.

`dotnet run --project bench/PdfPecker.Benchmarks -c Release -- --requirements [your.html ...]` measures this on your own machine and documents.

Usage

```
// String to bytes
byte[] pdf = HtmlToPdf.Convert(html);

// String to a stream
using (var file = File.Create("out.pdf")) HtmlToPdf.Convert(html, file);

// ASP.NET Core
app.MapGet("/invoice", () => Results.File(HtmlToPdf.Convert(html), "application/pdf",
"invoice.pdf"));

// File to file; relative  and <link> paths resolve against the HTML file's folder
HtmlToPdf.ConvertFile("invoice.html", "invoice.pdf");

// Off the calling thread
byte[] pdf = await HtmlToPdf.ConvertAsync(html);
```

All methods are static and thread-safe, so you can convert many documents in parallel.

Markdown

```
// Markdown to PDF, styled like a GitHub README
byte[] pdf = MarkdownToPdf.Convert(File.ReadAllText("README.md"));

// File to file; relative image paths resolve against the Markdown file's folder
MarkdownToPdf.ConvertFile("README.md", "README.pdf", new PdfOptions { FooterText = "Page
{page} of {pages}" });

// Markdown to HTML: a body fragment, or a complete page with the stylesheet
string fragment = MarkdownToHtml.Convert("# Hello\n\n*World*");     // <h1
id="hello">Hello</h1>\n<p><em>World</em></p>
string page = MarkdownToHtml.ConvertDocument(markdown);
MarkdownToHtml.ConvertFile("README.md", "README.html");
```

`MarkdownOptions` switches the GitHub extensions off for strict CommonMark (`GitHubExtensions = false`), drops heading ids (`HeadingIds = false`), shows raw HTML as text (`AllowRawHtml = false`), replaces the stylesheet (`StyleSheet`, see `MarkdownToHtml.DefaultStyleSheet`) or sets the title (default: the first heading). `MarkdownToPdf` takes `PdfOptions` and `MarkdownOptions` side by side.

Options

```
var options = new PdfOptions
{
    PageSize = PageSize.Letter.Landscape,            // A3, A4 (default), A5, Letter, Legal or
new PageSize(w, h) in points
    Margins = PageMargins.FromMillimeters(20, 15, 20, 15),
    Scale = 1,                                       // like Chrome's print scale (see its
docs to match IronPDF's output)
    TruncateBandsToWholePoints = false,              // round header/footer height attributes
down to whole points, as Chrome does
    BaseDirectory = "/var/app/templates",            // where relative image/stylesheet paths
are resolved
    ResourceResolver = url => myCache.Get(url),      // serve http(s) or embedded resources
yourself; null = default handling
    UserStyleSheet = "body { font-size: 10pt }",     // extra CSS applied after the
document's own
    FooterText = "Page {page} of {pages}",           // also HeaderText
    Title = "Invoice 42", Author = "Accounts",       // document properties (Title defaults
to <title>)
    Keywords = "invoice", Creator = "MyApp",         // also Subject, and Producer (default
"PdfPecker")
    CustomProperties = { ["RunId"] = "42" },         // your own properties, in the PDF's
document information
    Bookmarks = true,                                // PDF bookmarks from h1-h6 (default)
    Compress = true,                                 // Flate-compress page content (default)
    AllowFileAccess = false,                         // true: also read files outside
BaseDirectory (trusted HTML only)
    MaxPages = 1000,                                 // refuse longer documents with
PdfLimitException (default 100,000)
};
byte[] pdf = HtmlToPdf.Convert(html, options);
```

Local images and stylesheets are read only from inside `BaseDirectory` (the input file's folder for `ConvertFile`): absolute paths, `file:` URLs, `..` and symbolic links that lead outside it are ignored

unless `AllowFileAccess = true`, and converting a string without a `BaseDirectory` reads no files at all. For HTML from untrusted users, keep it that way, set a low `MaxPages`, and pass a cancellation token with a timeout (`HtmlToPdf.Convert(html, options, token)`): cancelling stops the conversion where it is.

A CSS `@page { size: A4 landscape; margin: 15mm }` rule overrides the page size and margins unless `IgnoreCssPageRules` is set. PdfPecker never makes network requests on its own: `http(s)` images and stylesheets are only loaded through `ResourceResolver`.

Fonts

Register TrueType or OpenType fonts and use them by name in CSS. Only the glyphs a document uses are embedded, and text can use any Unicode character the fonts have (Greek, Cyrillic, Central European, CJK, ...):

```
var options = new PdfOptions();
options.Fonts.AddFile("fonts/OpenSans-Regular.ttf")
             .AddFile("fonts/OpenSans-Bold.ttf");            // family, weight and style are
read from the files
options.Fonts.Add(bytes, familyName: "sans-serif");          // or override a generic family
options.Fonts.AddDirectory(@"C:\Windows\Fonts");              // read once, loaded only when
used
byte[] pdf = HtmlToPdf.Convert("<p style=\"font-family: 'Open Sans'\">Łódź, Ελλάδα,
Москва</p>", options);
```

Documents can bring their own fonts with `@font-face` (files, `data:` URIs or `ResourceResolver` URLs; `local()` refers to registered fonts):

```
@font-face { font-family: Brand; src: url(fonts/brand.woff2) format("woff2"); font-weight:
400; }
@font-face { font-family: Brand; src: url(fonts/brand-bold.ttf); font-weight: 700; unicode-
range: U+0000-024F; }
```

A `FontCollection` can be shared between options and parallel conversions. Formats: `.ttf`, `.otf`, `.ttc` collections, WOFF and WOFF2 (including WOFF2 collections; decoded with .NET's built-in Brotli). Fonts are matched like browsers do: by family, then style, then the nearest weight; a missing bold or italic face is synthesized. A character that the chosen font lacks is drawn with the next font in the `font-family` list that has it, then the standard font, then any other registered font.

Text without a registered font uses the standard PDF fonts (Helvetica, Times, Courier), which viewers supply themselves, so files stay small but are limited to Western European characters. Set `EmbedStandardFonts = true` to embed the bundled Liberation fonts in their place instead: they have the same widths, so the layout does not change, the text looks the same in every viewer, and any character they cover (Latin, Greek, Cyrillic, ...) works. PDF/A always does this. It adds a subset of each font used to the file (typically 10-25 KB per font).

A family that no font provides falls back silently, like in a browser. To notice, set `MissingFont`: it is called once per conversion for each family that had to be substituted, and throwing from it fails the conversion. To use the fonts installed on the machine for families you have not registered, set `UseSystemFonts = true` (off by default, so the output does not depend on the machine):

```
options.MissingFont = family => logger.LogWarning("Font {Family} is not available", family);
options.UseSystemFonts = true;   // looks in C:\Windows\Fonts, /Library/Fonts,
/usr/share/fonts, ...
```

Watermarks, passwords and PDF/A

```
var options = new PdfOptions
{
    // A diagonal stamp on every page. Also: Image (PNG, JPEG, GIF or WebP bytes), FontSize,
LetterSpacing, Angle, Opacity, Bold, Behind.
    Watermark = new PdfWatermark { Text = "WATERMARK", Color = "#c00000", Opacity = 0.25f },

    // AES-256 encryption (PDF 2.0 / Acrobat X and later). Without an OwnerPassword a random
one is used,
    // so the permissions cannot be lifted.
    Security = new PdfSecurity
    {
        UserPassword = "1234",                                  // needed to open the file;
null = opens freely
        OwnerPassword = "owner-secret",                         // full access
        Permissions = PdfPermissions.Print | PdfPermissions.Copy,
    },
};
```

For archiving, set `Conformance = PdfConformance.PdfA2b` (or `PdfA3b`): the PDF gets XMP metadata, an sRGB output intent and a file ID, and CMYK JPEGs are converted to RGB. All fonts are embedded: text without a registered font uses the bundled Liberation fonts (see Fonts). PDF/A does not allow encryption, so combining `Conformance` with `Security` throws an `ArgumentException`. AES-256 keys come from a password hash that is slow on purpose, which adds about 3 ms to a conversion.

Accessibility: tagged PDF and PDF/UA

```
var options = new PdfOptions
{
    Accessibility = PdfAccessibility.PdfUA1,   // or Tagged: the same structure without the
PDF/UA claim
    Language = "da",                           // default: <html lang="..."> for PDF/UA
without either, "en"
    Conformance = PdfConformance.PdfA2b,       // optional: PDF/UA combines with PDF/A
};
```

`Tagged` gives the PDF a structure tree built from the HTML, so screen readers, reflow and text extraction follow the document rather than the page layout: headings (`H1` to `H6`), paragraphs, lists with their labels, tables with header scopes and spans, figures with their alt text, links (tied to their clickable areas), block quotes, code and sections, all in document order, with `lang` attributes as the language of the element. Text outside a paragraph (straight in a `div`, say) gets one. Backgrounds, borders, running headers and footers, `HeaderText` / `FooterText`, watermarks, repeated table headers, fixed boxes and decorative images are artifacts that assistive technology skips.

`PdfUA1` adds what ISO 14289-1 asks beyond the tags: every font embedded (the bundled Liberation fonts stand in for the standard ones), the title shown in the viewer's title bar and the PDF/UA identification in the XMP metadata. The output passes veraPDF's PDF/UA-1 checks, on its own and

together with PDF/A-2b or PDF/A-3b (e-invoices included). It needs a title: `Title`, the `<title>` element or else the first heading; without one, conversion throws `ArgumentException`, as it does when `Security` leaves out `PdfPermissions.ExtractForAccessibility`.

What the tags can't fix is up to the HTML: give images an `alt` (an image without `alt`, `aria-label` or `title`, or an `<svg>` without a `<title>`, counts as decorative), use real headings, lists and `th` cells, and mark purely visual content `aria-hidden="true"`. Heading levels that skip one (`h1` then `h3`) are tagged one level below the previous heading, as PDF/UA does not allow gaps.

E-invoices (Factur-X / ZUGFeRD)

Pass the invoice's CII XML (from your invoicing system) and PdfPecker attaches it to the PDF, making a Factur-X / ZUGFeRD e-invoice: the pages are the human-readable invoice and the XML is what accounting software reads.

```
var options = new PdfOptions
{
    EInvoice = new PdfEInvoice(File.ReadAllBytes("invoice.xml")),
};
File.WriteAllBytes("invoice.pdf", HtmlToPdf.Convert(invoiceHtml, options));
```

The file is PDF/A-3b, so `Conformance` is raised to `PdfA3b` when it is left at `None` (`PdfA2b` or `Security` throw). The XML is embedded unchanged as `factur-x.xml` (`xrechnung.xml` for XRechnung), listed as an associated file, and the XMP metadata gets the Factur-X extension schema with the document type, version and profile. The profile (MINIMUM, BASIC WL, BASIC, EN 16931, EXTENDED or XRECHNUNG) is read from the XML's guideline ID; pass it as the second constructor argument to set it yourself. The relationship to the pages defaults to `Data` for MINIMUM and BASIC WL and `Alternative` otherwise; set `Relationship` to change it. PdfPecker checks that the XML is a well-formed Cross Industry Invoice (UBL is not allowed in Factur-X) but does not validate its content or build it from invoice data: make sure the visible pages show the same invoice. The output passes the Mustang validator (PDF/A-3b via veraPDF, and the XML's schematron).

PDF attachments in UBL e-invoices (Peppol, OIOUBL)

Peppol BIS Billing 3.0, OIOUBL, EHF, XRechnung UBL and other UBL e-invoices are XML only: the XML is the invoice, and the PDF travels inside it as an attachment. `UblInvoice.AttachPdf` adds one to the invoice or credit note your invoicing system made:

```
byte[] pdf = HtmlToPdf.Convert(invoiceHtml);
byte[] xml = UblInvoice.AttachPdf(File.ReadAllBytes("invoice.xml"), pdf, "INV-1001.pdf");
```

The PDF goes in as an `AdditionalDocumentReference` with `EmbeddedDocumentBinaryObject` (`mimeCode="application/pdf"` and the file name), after any attachments already there and where the UBL schema expects it. The rest of the XML, including its indentation, encoding and declaration, is kept as it was. The reference ID defaults to the file name; pass `id` and `description` to set them. For OIOUBL 2.1 (UBL 2.0) the description is written as the `DocumentType` OIOUBL requires, `PDF` by default. The output passes the UBL schemas, the Peppol BIS 3.0 and EN 16931 schematrons, and the OIOUBL 2.1 schematron. PdfPecker does not build or validate the rest of the invoice.

Digital signatures

Sign the PDF with a certificate (a `.pfx` file or one from the certificate store). Adobe Reader then shows who signed it and whether the file was changed afterwards.

```
var cert = X509CertificateLoader.LoadPkcs12FromFile("signer.pfx", "pfx-password");
var options = new PdfOptions
{
    Signature = new PdfSignatureOptions(cert)
    {
        Reason = "Approved",
        Location = "Copenhagen",
        // Optional: a visible box (points from the top-left corner; Page = -1 is the last
page).
        Appearance = new PdfSignatureAppearance { Page = -1, X = 340, Y = 720, Width = 220,
Height = 70 },
        // Optional: a trusted RFC 3161 timestamp (PAdES B-T). The only option that uses the
network.
        TimestampUrl = "http://timestamp.digicert.com",
    },
};

// Or sign any existing PDF; earlier signatures stay valid.
byte[] signed = PdfSigner.Sign(File.ReadAllBytes("in.pdf"), new PdfSignatureOptions(cert));
```

Signatures are PAdES baseline (`ETSI.CAdES.detached`) by default, or classic `adbe.pkcs7.detached` with `Format = PdfSignatureFormat.Pkcs7Detached`; RSA and ECDSA keys with SHA-256/384/512. The signature is appended as an incremental update, and it works together with `Security` (encrypted files) and `Conformance` (PDF/A: the visible box is drawn as outlines, so it needs no unembedded font). The CMS structure is built with .NET's own `System.Formats.Asn1` and `System.Security.Cryptography`; no extra package is needed. Signing adds a few milliseconds (the RSA or ECDSA operation and re-reading the file); conversions without a signature are unaffected.

Custom properties are written to the document information dictionary, where Acrobat shows them under Document Properties > Custom and `pdfinfo -custom` lists them. For PDF/A they are also written to the XMP metadata (Acrobat's `pdfx` namespace, with the extension schema PDF/A requires), alongside Keywords, Creator and Producer.

Merging PDFs

`PdfMerger` combines PDFs into one, any PDFs, not only those PdfPecker made:

```
byte[] merged = PdfMerger.Merge(invoicePdf, termsPdf, attachmentPdf);

// With passwords for encrypted inputs (by position), and properties for the result.
byte[] merged2 = PdfMerger.Merge(new[] { a, b }, new PdfMergeOptions
{
    Passwords = new string?[] { null, "secret" },
    Title = "Payslips October",
    CustomProperties = { ["RunId"] = "42" },
});

PdfMerger.MergeFiles("all.pdf", new[] { "a.pdf", "b.pdf" });

// Streams in and out: inputs are read one at a time and the result is written as it is made.
PdfMerger.Merge(new[] { invoiceStream, termsStream }, outputStream);
await PdfMerger.MergeAsync(uploads.Select(f => f.OpenReadStream()), response.Body);
```

With streams (and `MergeFiles`), only one input is held in memory at a time and the merged document goes to the output as each input is done, so memory follows the largest input rather than the whole batch: merging 100 files of 32 MB in total peaks at 14 MB of managed memory, against 105 MB with byte arrays. `MergeAsync` reads and writes asynchronously, as ASP.NET Core requires for request and response bodies. Input streams are read from their current position and are not disposed; the sequence is enumerated lazily, so an iterator can open each one only when its turn comes. `MergeFiles` writes to a temporary file and moves it into place, so the output can be one of the inputs and a failed merge leaves no partial file behind.

Merging PDFs as they arrive

For PDFs that arrive one at a time from elsewhere, such as downloads read a few at a time, use `StreamingPdfMerger`. It writes each PDF's pages when it is added and then lets the PDF go, so memory stays at about one input, and between inputs you decide what comes next: skip a missing document, or check that anything was added before answering. The output can be an HTTP response body or a file; it does not need to be seekable.

```
var merger = new StreamingPdfMerger(response.Body, new PdfMergeOptions { Title = "Payslips
October" });
foreach (var path in paths)
{
    var pdf = await storage.DownloadAsync(path, ct);
    if (pdf.Length == 0) continue;          // skip a missing document
    await merger.AddAsync(pdf, ct);
}
if (merger.InputCount == 0) return Results.NotFound();
await merger.CompleteAsync(ct);             // writes the page tree, bookmarks, form and
cross-reference table

// Or, from an IAsyncEnumerable<byte[]>:
int pages = await PdfMerger.MergeAsync(DownloadInOrder(paths), response.Body,
cancellationToken: ct);
```

The result is the same as `PdfMerger.Merge`'s, but the output is only a valid PDF once `Complete` has run. If adding a PDF throws (a damaged or encrypted input, `MaxPages` reached), what was written cannot be used and the merger refuses further calls. `Passwords` are matched to inputs in the order they are added.

The pages keep their content, fonts, images, links, annotations, layers and fillable form fields, and each input's bookmarks follow each other. A form field named like one in an earlier input is renamed `name_2`, `name_3` and so on, so the copies keep their own values. Document properties come from the first input unless `PdfMergeOptions` sets them. The result is never encrypted and has no signatures (merging would break them); PDF/A conformance, attachments, page labels and tags are not carried over. Images, fonts and other resources that are byte-for-byte the same in several inputs, such as a logo on every payslip of a run, are written once.

Inputs are handled as untrusted: damaged files are repaired where possible, decoding is limited per input, and `MaxPages` (default 100,000) caps the result. Inputs without any pages throw `InvalidDataException`. Merging was tested on PDFs from Chrome (as IronPDF makes them), Ghostscript, LibreOffice, MuPDF, qpdf, ReportLab, pypdf incremental updates, scanners (JPEG, CCITT G4, CMYK, OCR text layers) and damaged files (wrong or missing cross-reference tables, truncated files, junk before the header); every merged page rendered pixel-identical to its original in Poppler. Merging four small documents takes about 1 ms, and 1,000 one-page Chrome payslips about 1 s.

PDF to image

`PdfToImage` and `PdfRenderer` render PDF pages to PNG (or raw RGBA pixels). This works for any PDF, not only PdfPecker's own output, and also uses no third-party code: the PDF parser, font engines, JPEG and fax decoders, rasterizer and PNG encoder are all part of the library.

```
// Every page as PNG bytes (rendered in parallel), or one page (0-based index)
byte[][] pages = PdfToImage.Convert(pdfBytes, new ImageOptions { Dpi = 150 });
byte[] first = PdfToImage.ConvertPage(pdfBytes, 0);

// report.pdf -> out/report-1.png, out/report-2.png, ...
PdfToImage.ConvertFile("report.pdf", "out");

// Open once, render many pages; RenderedImage also exposes the RGBA pixels
var renderer = PdfRenderer.Open(pdfBytes, password: null);
RenderedImage image = renderer.RenderPage(2, new ImageOptions { Width = 800 });
image.SavePng("page3.png");
```

`ImageOptions` sets `Dpi` (default 150) or a target `Width`/`Height` in pixels, `BackgroundColor` or `Transparent`, `RenderAnnotations`, `Password`, `MaxDegreeOfParallelism` and `PngCompression`. A `PdfRenderer` is thread-safe and shares decoded fonts and images between pages. Encrypted files open with the user or owner password (`PdfPasswordException` otherwise); unreadable files throw `InvalidDataException`, as does a page that needs more than 20 million drawing operations (nested forms can multiply a few bytes into billions). Pass a cancellation token to bound the time a page may take.

What the renderer supports:

- **Files**: cross-reference tables and streams, object streams, incremental updates, damaged files (rebuilt by scanning), RC4 and AES-128/256 encryption, all standard filters (Flate, LZW, ASCII85, ASCIIHex, RunLength, predictors).
- **Graphics**: paths, fills (non-zero and even-odd), strokes with joins, caps and dashes, clipping, anti-aliasing, all blend modes, constant alpha, soft masks, transparency groups, axial, radial,

function and mesh shadings (types 1 to 7), tiling patterns, form XObjects, optional content (layers) and annotation appearances.

- **Text**: embedded TrueType, OpenType, CFF and Type 1 fonts, Type 3 fonts, CID fonts with CMaps, vertical text, all text rendering modes. The 14 standard fonts and non-embedded fonts are drawn with built-in Liberation and DejaVu-derived faces stretched to the document's widths.
- **Images**: every bit depth and colour space (Gray, RGB, CMYK, Lab, ICC-based, Indexed, Separation, DeviceN), baseline and progressive JPEG, JPEG 2000 (`JPXDecode`, including embedded alpha), JBIG2 (generic, text, symbol, refinement and halftone regions, with `JBIG2Globals`), CCITT fax, stencil masks, colour-key masks and soft masks.

For what the renderer does not support yet, see Limitations.

What is supported

HTML: the common document elements: headings, paragraphs, `div` / `section`-style blocks, `span`, `a` (external links and `#id` links inside the document), `b` / `strong`, `i` / `em`, `u`, `s`, `sub`, `sup`, `small`, `big`, `mark`, `code`, `pre`, `blockquote`, `hr`, `br`, `ul` / `ol` / `li` (nested, with `start` and `value`), `dl`, `table` (with `thead`, `tbody`, `tfoot`, `caption`, `colspan`, `rowspan`, `col width`), `img`, inline `<svg>`, and legacy attributes such as `align`, `valign`, `bgcolor`, `width`, `height`, `border`, `cellpadding`, `cellspacing` and `<font>`. The parser is forgiving about unclosed and misnested tags, the way browsers are.

Markdown: CommonMark 0.31 (651 of the 652 spec examples pass; the one miss is an exotic HTML5 entity name) plus the GitHub Flavored Markdown extensions: tables with column alignment, `~~strikethrough~~`, task list items (`- [x]`) and bare `www.` / `https://` / e-mail autolinks. Headings get GitHub-style ids, so `[see below](#heading-text)` links work in the PDF too. The parser runs in linear time on hostile input (deep nesting, unclosed brackets, long delimiter runs). See Limitations for what is not supported.

CSS: `<style>` blocks, `<link rel="stylesheet">`, inline `style` attributes, `@media print`, and `@page`. Selectors: type, `.class`, `#id`, `*`, attribute selectors (`[a]`, `=`, `~=`, `|=`, `^=`, `$=`, `*=`), descendant, `>`, `+`, `~`, `:first-child`, `:last-child`, `:only-child`, `:nth-child()`, `:nth-last-child()`, `:first-of-type`, `:last-of-type`, `:only-of-type`, `:nth-of-type()`, `:nth-last-of-type()`, `:not()`, `:is()` and `:where()` (all three with selector lists; `:is()` and `:where()` skip entries they do not understand), `:root`, `:link`, `:empty`, with specificity and `!important` handled as in browsers. Properties:

Area	Properties
Box model	`display` (block, inline, inline-block, flex, inline-flex, grid, inline-grid, list-item, table parts, none), `margin`, `padding`, `border` (solid, dashed, dotted, double), `border-radius` (incl. elliptical `/` radii and percentages; rounds backgrounds, borders, shadows and images), `box-shadow` (outer shadows; a blur is drawn as fading bands), `overflow` (`hidden`, `clip`, `auto` and `scroll` clip to the padding box, following its rounded corners, also in table cells; `overflow-x` / `overflow-y`), `outline` and `outline-offset` (all border styles; follows rounded corners), `width`, `height`, `min-` / `max-width`, `min-height`, `box-sizing`, `aspect-ratio` (incl. `auto <ratio>`), `object-fit` and `object-position` on images

Area	Properties
Text	`color`, `font`, `font-family`, `font-size`, `font-weight`, `font-style`, `line-height`, `text-align` (incl. justify), `text-decoration`, `text-transform`, `text-indent`, `letter-spacing`, `word-spacing`, `white-space`, `overflow-wrap` / `word-wrap`, `word-break` (`break-all`), `vertical-align`, `text-shadow` (several shadows; a blur is drawn as faint copies of the text), `text-overflow: ellipsis` (on a box that clips its overflow). Lines break as in Chrome: at spaces, after hyphens and dashes, at `` (showing a hyphen) and ``; a word too wide for its line overflows unless `overflow-wrap` or `word-break` allows breaking it. `line-height: normal` comes from the font's own metrics, rounded to whole pixels as Chrome does, and border widths snap to whole pixels. Text is kerned as in Chrome: from the font's GPOS or `kern` table, and for the standard fonts from the Liberation fonts browsers use in their place. Embedded fonts get their standard ligatures (GSUB `liga` / `clig`, such as fi and ffi) as in Chrome, except in letter-spaced text; `font-variant-ligatures: none` or `font-feature-settings: "liga" 0` turns them off
Backgrounds	`background-color`, `background-image` and `background` with `url()` images (PNG, JPEG, GIF, WebP, and SVG files drawn as vectors), `linear-gradient()`, `radial-gradient()`, `repeating-linear-gradient()` and `repeating-radial-gradient()` (several layers allowed); `background-repeat` (incl. `space`, `round`), `background-position` (incl. edge offsets like `right 10px bottom 5px`), `background-size` (`cover`, `contain`, lengths), `background-origin`, `background-clip`. Repeated images are written once as PDF tiling patterns. `url()` in a linked stylesheet is relative to the stylesheet and follows the same file access rules as ``. As in Chrome's print output, the background of `<html>`, or else of ` `, fills every page area (images and gradients too, positioned against the root box and continuing from page to page); `@page { background: }` also colours the page margins. `opacity` on SVG
Flexbox	`flex-direction`, `flex-wrap`, `flex-flow`, `flex`, `flex-grow` / `-shrink` / `-basis`, `order`, `justify-content`, `align-items`, `align-self`, `align-content`, `gap`, `margin: auto`
Grid	`grid-template-columns` / `-rows` (lengths, `%`, `fr`, `auto`, `min-content`, `max-content`, `minmax()`, `fit-content()`, `repeat()` incl. `auto-fill` / `auto-fit`), `grid-auto-rows` / `-columns`, `grid-auto-flow`, `grid-template-areas`, `grid-template`, named lines (`[name]`), `grid-column` / `-row` (line numbers, names and `span`), `grid-area` (incl. area names), `gap`, `justify-items` / `-self`, `align-items` / `-self`, `justify-content`, `align-content`, `place-*`
Columns	`column-count`, `column-width`, `columns`, `column-gap`
Variables	custom properties (`--name: value`) and `var(--name, fallback)`, anywhere including shorthands
Math functions	`calc()`, `min()`, `max()` and `clamp()` for lengths and numbers, nested in any combination and mixing units (`calc(100% - 2em)`, `min(100%, 600px)`, `clamp(1rem, 2.5vw, 2rem)`), with `var()` inside them. Percentages resolve against the property's own reference (the containing block's width for `width` and margins, the parent's font size for `font-size`), so they work in widths, heights, margins, padding, flex bases, grid tracks, gaps, positions, translations, font sizes and borders
Positioning	`float` and `clear`; `position` (`relative`, `absolute`, `fixed`; `sticky` is drawn as `relative`) with `top`, `right`, `bottom` and `left`; `z-index`
Transparency	`opacity` (a block is painted as one PDF transparency group; an inline `<span>` fades its text and background), translucent colours everywhere (`rgba()`, `hsla()`, `#rrggbbaa`, `transparent`), gradients that fade to `transparent` (as a soft mask). PDF/A-2 and 3 allow all of this
Transforms	`transform` (`translate()`, `translateX()` / `Y()`, `scale()`, `scaleX()` / `Y()`, `rotate()`, `skew()`, `skewX()` / `Y()`, `matrix()`) and `transform-origin`
Generated content	`::before` and `::after` with `content` (strings, `attr()`, `counter()`, `counters()`, `open-quote` / `close-quote`), `counter-reset`, `counter-increment`, `counter-set`

Area	Properties
SVG	`fill`, `stroke`, `stroke-width`, `fill-opacity`, `stroke-opacity`, `fill-rule`, `stroke-linecap`, `stroke-linejoin`, `stroke-miterlimit`, `clip-path`, `clip-rule`, `mask`, `mask-type`, `filter`, `marker`, `marker-start` / `-mid` / `-end`, `flood-color`, `flood-opacity`, `color-interpolation-filters`, `text-anchor`, `dominant-baseline` (as CSS or presentation attributes)
Lists	`list-style-type` (disc, circle, square, decimal, decimal-leading-zero, alpha, roman, none), `list-style`
Tables	`border-collapse` (each shared border drawn once, the wider one winning), `border-spacing`, cell `vertical-align`, automatic column widths, `table-layout: fixed`, a table `height` shared out over the rows
Paging	`page-break-before` / `after` / `inside`, `break-before` / `after` / `inside` (`always`, `page`, `avoid`), `orphans`, `widows`, `@page` size and margins

Units: `px`, `pt`, `em`, `rem`, `%`, `mm`, `cm`, `in`, `pc`, `ex`, `ch`, `vw`, `vh`, `vmin`, `vmax` (the viewport is the page's content area, minus any running header and footer, in whole pixels as in Chrome). Colours: names, `#rgb`, `#rrggbb`, `#rrggbbaa`, `rgb()`, `rgba()`, `hsl()`, `hsla()`.

Pagination: lines, images and table rows are never cut in half at a page boundary; table header rows repeat on every page a table spans; backgrounds and borders continue across pages. Flex lines and grid rows move to the next page as a whole when they don't fit. A paragraph that splits leaves at least `orphans` lines (default 2) at the bottom of a page and carries at least `widows` lines (default 2) to the next. Headings keep with the content that follows them (`break-after: avoid` in the default styles), and `break-before` / `break-after: avoid` does the same for any block. `break-inside: avoid` keeps a block or a whole table on one page when it fits.

Links and bookmarks: `<a href>` becomes a clickable link, whether inline or a block (a card wrapped in a link), and stays clickable on both pages when a page break splits it. External URLs open in the viewer's browser and `#id` links jump to the element. Headings `h1`-`h6` become the PDF's bookmarks, nested by level; set `PdfOptions.Bookmarks = false` to leave them out.

Running header and footer: an element with the `data-sticky-pdf-header` or `data-sticky-pdf-footer` attribute is taken out of the flow and drawn at the top or bottom of every page, and the page content flows in the space between. Inside it, `{page}` and `{total-pages}` (or `{pages}`) are replaced with the page number and page count. An optional `height` attribute (`height="17mm"`) reserves at least that much space.

```
<header data-sticky-pdf-header height="17mm">ACME Ltd · Payslip</header>
<main>...</main>
<footer data-sticky-pdf-footer height="12mm">Page {page} of {total-pages}</footer>
```

SVG: inline `<svg>` and SVG files (in `` and as `url()` backgrounds, including `data:image/svg+xml` URIs) with `path` (all commands, arcs included), `rect` (incl. rounded), `circle`, `ellipse`, `line`, `polyline`, `polygon`, `text` and `tspan` (`x`, `y`, `dx`, `dy`, `text-anchor`, `dominant-baseline`, standard fonts), `g`, `use`, nested `svg`, `viewBox`, `preserveAspectRatio` and `transform`, drawn as vector paths. `linearGradient` and `radialGradient` (both units, `gradientTransform`, `spreadMethod`, `href` inheritance) become PDF shadings, and `pattern` fills and strokes (both units, `viewBox`, `patternTransform`, `href` inheritance) become PDF tiling patterns. `clipPath` clips the elements that reference it, and `mask` (luminance or alpha, both units) becomes a PDF soft mask. `marker` draws

arrowheads and dots at the vertices of paths, lines, polylines and polygons (`orient` incl. `auto` and `auto-start-reverse`, `markerUnits`, `viewBox`, `refX`/`refY`, clipped to the marker unless `overflow="visible"`). `filter` is applied in pixels, at 144 dpi, and drawn as an image: `feGaussianBlur`, `feOffset`, `feDropShadow`, `feFlood`, `feMerge`, `feComposite` (all operators), `feBlend` (separable modes), `feColorMatrix`, `feComponentTransfer` and `feMorphology`, in linearRGB or sRGB. As in browsers, an SVG file is styled only by its own `